

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

«Контейнеризация написанного приложения средствами Docker»

Выполнила:

Гайдук Алина Сергеевна

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках лабораторной работы №3 я контейнеризировала backend-приложение RecipeHub, которое было реализовано в ЛР2 как набор микросервисов. Требовалось подготовить Dockerfile для каждого сервиса, описать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами внутри Docker-сети.

Для выполнения работы я использовала исходный код ЛР2 и вынесла результат в отдельный каталог labs/lab3. Такой подход позволяет сохранить предыдущую лабораторную работу без изменений и одновременно получить самостоятельный проект для проверки контейнеризации.

Ход работы

Подготовка структуры проекта

Приложение состоит из шести запускаемых Go-приложений: gateway, identity-service, catalog-service, recipe-service, blog-service и engagement-service. Для каждого сервиса сохранен отдельный entrypoint в cmd/<service>, а контейнеризация не меняет бизнес-логику приложения.

Сервис	Назначение	Dockerfile
gateway	Единая публичная точка входа и reverse-проху на сервисы-владелец.	docker/gateway/Dockerfile
identity-service	Регистрация, авторизация, профили пользователей и подписки.	docker/identity-service/Dockerfile
catalog-service	Справочники рецептов и проверка reference id.	docker/catalog-service/Dockerfile

recipe-service	Рецепты, шаги, ингредиенты, теги и проекции активности.	docker/recipe-service/Dockerfile
blog-service	Посты блога и проекции активности по постам.	docker/blog-service/Dockerfile
engagement-service	Комментарии, лайки, сохраненные рецепты и публикация событий.	docker/engagement-service/Dockerfile

Dockerfile для каждого сервиса

Для каждого сервиса я создала отдельный Dockerfile в каталоге `docker<service`. Все Dockerfile построены по одной схеме, потому что сервисы собираются из одного Go-модуля и отличаются только путем к `entrypoint`. При этом требование лабораторной работы выполняется буквально: у каждого сервиса есть собственный файл сборки.

Сборка выполнена как `multi-stage build`. На первом этапе используется образ `golang:1.23-bookworm`, загружаются зависимости из `go.mod` и `go.sum`, затем собирается конкретный бинарный файл из `./cmd/service`. Переменная `CGO_ENABLED` установлена в 0, поэтому итоговый бинарник не зависит от C-библиотек. На втором этапе используется `debian:bookworm-slim`; в него копируется только собранный бинарник, файл `docs/openapi.yaml` и минимальные runtime-зависимости `ca-certificates` и `wget`.

В runtime-образе приложение запускается не от `root`, а от системного пользователя `appuser`. Для каждого контейнера задан `EXPOSE 8080` и `healthcheck` на `/healthz`. Это позволяет `docker-compose` отслеживать готовность сервисов и запускать зависимые контейнеры только после успешной проверки.

Общий docker-compose.yml

Общий `docker-compose.yml` описывает весь стек приложения: PostgreSQL, RabbitMQ и шесть Go-сервисов. Для всех сервисов задана общая

bridge-сеть `recipehub_net`. Благодаря этому контейнеры обращаются друг к другу по DNS-именам сервисов Docker Compose: `identity-service`, `catalog-service`, `recipe-service`, `blog-service`, `engagement-service`, `postgres` и `rabbitmq`.

Наружу публикуются только необходимые порты: `gateway` доступен на `RECIPEHUB_PORT`, `PostgreSQL` – на `POSTGRES_PORT` для отладки и локального доступа, `RabbitMQ` – на стандартном AMQP-порту и на `management`-порту. Остальные микросервисы не публикуются наружу и доступны только внутри Docker-сети через `expose 8080`. Это соответствует идее единой публичной точки входа через `gateway`.

Контейнер	Сетевое имя	Роль
postgres	postgres:5432	Один общий контейнер PostgreSQL с отдельными базами для сервисов.
rabbitmq	rabbitmq:5672	Брокер событий для обмена интеграционными событиями.
gateway	gateway:8080	Публичный reverse-proxy, публикуется на порт хоста.

Настройка PostgreSQL

Для базы данных я использовала один общий контейнер PostgreSQL. Это не нарушает принцип `database-per-service` для лабораторной работы, потому что разделение выполнено на уровне отдельных баз данных: `recipehub_identity`, `recipehub_catalog`, `recipehub_recipe`, `recipehub_blog` и

recipehub_engagement. Они создаются при первом старте контейнера скриптом.

Каждый сервис получает собственную строку подключения вида `<SERVICE>_DATABASE_URL` и работает только со своей базой. Внутри Docker-сети в этих строках используется `host postgres` и порт `5432`, а не `127.0.0.1`. Это важно, потому что `localhost` внутри контейнера указывает на сам контейнер, а не на базу данных.

Переменные окружения

Я привела `.env` и `.env.example` к одному набору переменных, которые реально используются `docker-compose.yml`. В файлах оставлены параметры портов, тега образов, учетные данные PostgreSQL, строки подключения сервисов к своим базам, JWT-секреты, `SERVICE_TOKEN` и `EVENTS_EXCHANGE`.

В `docker-compose.yml` внутренние URL сервисов заданы напрямую через имена контейнеров. Это делает конфигурацию устойчивой: даже если пользователь скопирует `.env.example` в `.env`, контейнеры будут обращаться к PostgreSQL по `postgres:5432` и к RabbitMQ по `rabbitmq:5672`. Для `healthcheck` используется `127.0.0.1` внутри самого контейнера, и это корректно, так как проверяется локальный HTTP-сервер сервиса.

Сетевое взаимодействие сервисов

Сетевое взаимодействие построено через Docker DNS и общую сеть. Gateway получает переменные `IDENTITY_SERVICE_URL`, `CATALOG_SERVICE_URL`, `RECIPE_SERVICE_URL`, `BLOG_SERVICE_URL` и `ENGAGEMENT_SERVICE_URL` со значениями вида `http://identity-service:8080`. После этого он проксирует публичные маршруты `/api/v1` в сервис-владелец.

Внутренние HTTP-запросы между сервисами также идут по именам контейнеров. Например, `recipe-service` обращается к `identity-service` для проверки автора и к `catalog-service` для проверки справочников. `Engagement-service` обращается к `recipe-service` и `blog-service` для проверки цели лайка или комментария. Для защиты внутренних ручек используется `SERVICE_TOKEN`.

Nginx в данной реализации я не добавляла. Его роль в проекте уже выполняет `gateway`: он является единой публичной точкой входа и `reverse-proxy` для микросервисов. Добавление отдельного `nginx`-контейнера в этой лабораторной работе привело бы к дублированию функции `gateway` без новой пользы для требований задания.

Вывод

В ходе лабораторной работы я контейнеризировала микросервисное backend-приложение средствами Docker. Для каждого сервиса я подготовила отдельный `Dockerfile`, описала общий `docker-compose.yml` и настроила взаимодействие контейнеров через общую сеть Docker Compose.

В итоговой конфигурации `gateway` является единственной публичной точкой входа, а остальные сервисы доступны только внутри сети. PostgreSQL сведен в один инфраструктурный контейнер, но данные сервисов разделены по отдельным базам. RabbitMQ вынесен в отдельный контейнер и доступен сервисам по имени `rabbitmq`.